

Day 3, Session 2: Asynchronous Programming (`concurrent.futures` & `asyncio`)

Duration: 2 hours (theory + live demonstrations)

Prerequisites: Days 1 & 2 (Threading, Multiprocessing, Event loop fundamentals)

Demos folder: [day3/demo/](#)

Table of Contents

1. The Concurrency Landscape
 2. High-Level Pools with `concurrent.futures`
 - `ThreadPoolExecutor` vs `ProcessPoolExecutor`
 - Submitting Work: `submit()` vs `map()`
 - Working with Futures: `as_completed()`, Callbacks & Timeouts
 3. Cooperative Multitasking with `asyncio`
 - The Event Loop Model
 - Coroutines: `async def` and `await`
 - Task Management: `create_task()` and Cancellation
 - Concurrent Execution with `asyncio.gather()`
 4. Bridging `asyncio` with CPU-Bound & Blocking Code
 5. Choosing the Right Concurrency Tool
 6. Session 2 Summary & What's Next
-

1. The Concurrency Landscape

Over the past three days, we have encountered three different ways to handle work:

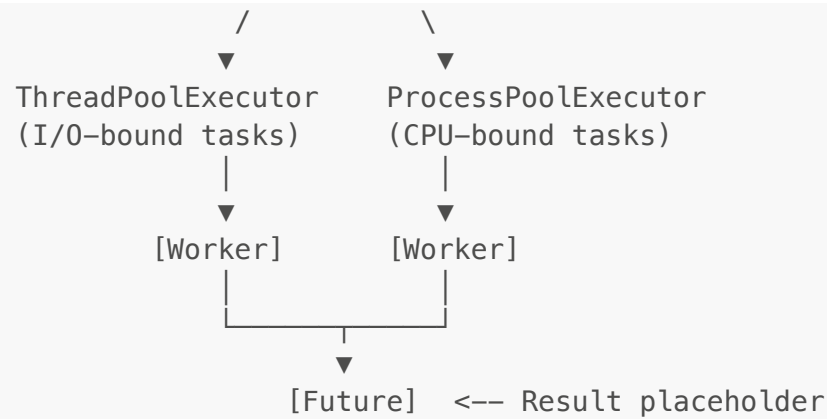
1. **Preemptive Multithreading** (`threading`): The OS scheduler switches between threads at will. Ideal for I/O, limited by GIL for CPU.
2. **Preemptive Multiprocessing** (`multiprocessing` / `MPI`): Separate OS processes with separate memory spaces. True parallelism on multi-core CPUs and distributed clusters.
3. **Cooperative Multitasking** (`asyncio`): A single thread switches tasks **voluntarily** when waiting for I/O. Extremely lightweight (handles 100,000+ connections without thread memory overhead).

In this session, we examine modern, high-level abstractions for both styles.

2. High-Level Pools with `concurrent.futures`

Introduced in Python 3.2 (PEP 3148), `concurrent.futures` provides a unified, elegant interface for executing asynchronous tasks in pools.

`concurrent.futures` Architecture
[Executor]



ThreadPoolExecutor vs ProcessPoolExecutor

Both inherit from the abstract base class `Executor`, meaning you can switch from threads to processes by changing **a single line of code**:

```

from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

# For I/O operations (network requests, disk writes):
with ThreadPoolExecutor(max_workers=8) as executor:
    ...

# For CPU-heavy math (bypassing the GIL):
with ProcessPoolExecutor(max_workers=8) as executor:
    ...
  
```

Submitting Work: `submit()` vs `map()`

`executor.map(func, iterable)`

- Submits all items in the iterable to the pool.
- Returns results in the **exact same order** as the input.
- Blocks as you iterate over the generator if earlier items are not yet ready.

`executor.submit(func, *args, **kwargs)`

- Submits an individual task and immediately returns a **Future** object.
- Non-blocking: your main thread continues executing.
- Allows tasks with different arguments or functions to be submitted independently.

```

with ThreadPoolExecutor() as executor:
    # Submit individual task -> returns Future immediately
    future = executor.submit(pow, 2, 32)
  
```

```
# Retrieve result (blocks until task is done)
result = future.result()
```

Working with Futures

A **Future** represents the eventual result of an asynchronous operation.

Key Future Methods

- `future.done()`: Returns **True** if the task has completed or was cancelled.
- `future.result(timeout=None)`: Returns the value returned by the worker. If an exception occurred inside the worker, `result()` **re-raises** that exception in the caller!
- `future.add_done_callback(fn)`: Attaches a callable that executes automatically as soon as the future completes.
- `future.cancel()`: Attempts to cancel the task if it hasn't started yet.

`as_completed(futures_list)`

Rather than waiting for futures in the order they were submitted, `as_completed()` yields futures **in the order they finish**:

```
from concurrent.futures import ThreadPoolExecutor, as_completed

with ThreadPoolExecutor(max_workers=4) as executor:
    futures = [executor.submit(fetch_url, url) for url in urls]

    for f in as_completed(futures):
        data = f.result() # process each result as soon as it arrives!
```

Demo Time

Now, let's see these theoretical topics in action.

- `03_concurrent_futures.py` —
 1. `demo_thread_pool()` — Fetching URLs concurrently with `ThreadPoolExecutor` and `as_completed()`
 2. `demo_process_pool()` — CPU-bound parallel math with `ProcessPoolExecutor.map()`
 3. `demo_future_lifecycle()` — Future callbacks, error propagation, and timeout handling

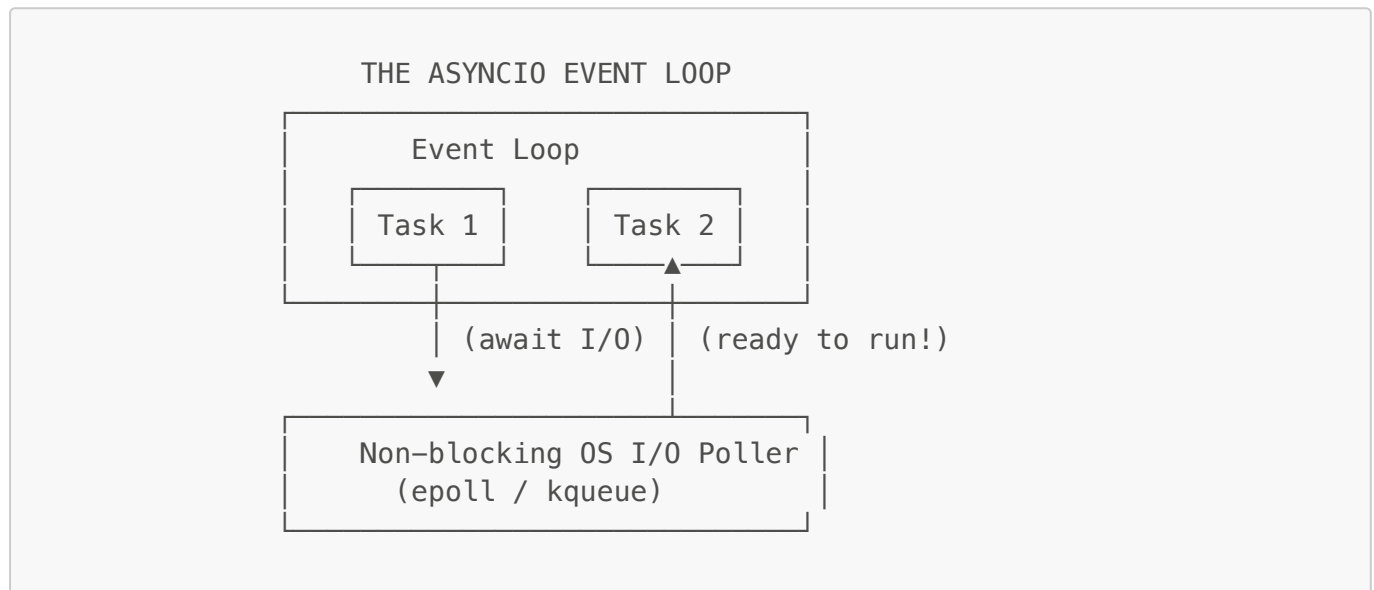
3. Cooperative Multitasking with **asyncio**

The Event Loop Model

Traditional threads use **preemptive scheduling**: the operating system arbitrarily interrupts your code to switch threads. This requires complex locks to prevent race conditions.

asyncio uses **cooperative multitasking** driven by an **Event Loop**:

- Everything runs inside a **single operating system thread**.
- When a task reaches an I/O wait (network packet, timer, database read), it explicitly **yields control** (**await**).
- While that task waits, the event loop runs other tasks that are ready.
- **Zero race conditions** on shared memory because only one piece of code executes at any exact microsecond!



Coroutines: **async def** and **await**

- **async def**: Declares a coroutine function. Calling it does not execute the function; it returns a **coroutine object**.
- **await**: Suspends the execution of the current coroutine until the awaited operation (another coroutine, task, or Future) finishes, returning control to the event loop.

```
import asyncio

async def greet(name, delay):
    print(f"Starting {name}...")
    await asyncio.sleep(delay) # Non-blocking pause
    print(f"Finished {name}!")
    return f"Done with {name}"

# Launch the top-level event loop:
asyncio.run(greet("World", 1.0))
```

To run coroutines concurrently, you schedule them onto the loop as a **Task** using `asyncio.create_task()`:

```
async def main():
    # Both tasks start executing immediately in the background
    task1 = asyncio.create_task(greet("Alice", 1.0))
    task2 = asyncio.create_task(greet("Bob", 0.5))

    # Wait for both to complete
    res1 = await task1
    res2 = await task2
```

Task Cancellation

Tasks can be cancelled cleanly by calling `task.cancel()`. The next time the task is awaited, an `asyncio.CancelledError` is raised inside the coroutine, allowing it to perform cleanups (closing files, sockets, etc.).

Concurrent Execution with `asyncio.gather()`

`asyncio.gather(*coros_or_tasks)` runs multiple awaitables concurrently and returns a list of results in the order they were submitted:

```
async def main():
    results = await asyncio.gather(
        fetch_user(1),
        fetch_user(2),
        fetch_user(3)
    )
    print("All users fetched:", results)
```

4. Bridging `asyncio` with CPU-Bound & Blocking Code

Golden Rule of Asyncio: NEVER run blocking I/O (like `time.sleep()`, synchronous `requests.get()`) or long CPU calculations inside a coroutine! It blocks the single event loop thread, freezing all other concurrent tasks!

If you must run a legacy blocking library or a CPU-intensive calculation, offload it to an executor using `loop.run_in_executor()`:

```
import asyncio
import time

def blocking_legacy_calc(n):
    time.sleep(2) # Synchronous blocking call
```

```

    return n * 2

async def main():
    loop = asyncio.get_running_loop()

    # Runs the blocking function in a background ThreadPool without
    # freezing asyncio!
    result = await loop.run_in_executor(None, blocking_legacy_calc, 21)
    print("Result:", result)

```

Demo Time

Now, let's see these theoretical topics in action.

- `04_asyncio_basics.py` —
 1. `demo_coroutines_and_gather()` — Concurrent coroutines with `asyncio.gather()`
 2. `demo_task_manipulation()` — Scheduling tasks with `create_task()` and cooperative cancellation
 3. `demo_asyncio_with_futures()` — Offloading blocking/CPU tasks using `run_in_executor`

5. Choosing the Right Concurrency Tool

Paradigm	Module	Best Suited For	Advantages	Drawbacks
Multithreading	<code>threading</code>	I/O-bound with shared memory	Direct variable sharing	Race conditions, GIL limits CPU speedup
Executor Pools	<code>concurrent.futures</code>	Batch tasks (I/O or CPU)	High-level API, Future abstraction	Less granular control over individual threads
Event Loop	<code>asyncio</code>	High-concurrency network I/O (10,000+ sockets)	Lightweight, single-threaded, zero race conditions	Requires all libraries to be async-compatible
Multiprocessing	<code>multiprocessing</code>	CPU-bound tasks on one computer	Bypasses GIL, utilizes all CPU cores	Higher memory overhead, requires IPC serialization
Message Passing	<code>mpi4py</code>	Multi-node supercomputing /	Scales to thousands of	Distributed memory only;

Paradigm	Module	Best Suited For	Advantages	Drawbacks
		HPC clusters	nodes	manual communication

Session 2 Summary & What's Next

Key Takeaways

- **concurrent.futures** provides an identical high-level API (**submit**, **map**, **as_completed**) for both threads and processes.
- **asyncio** delivers massive scalability for I/O concurrency through single-threaded cooperative multitasking.
- Use **run_in_executor()** whenever you need to marry async event loops with synchronous or CPU-heavy workloads.

What's Coming Tomorrow (Day 4)

Tomorrow we enter **Distributed Python**:

- Task queues with **Celery**
- Scientific distributed computing with **SCOOP**
- Remote Method Invocation with **Pyro4**
- Communicating Sequential Processes with **PyCSP**
- Remote Procedure Calls with **RPyC**

End of Day 3